

Docker Command Guide

for your Django + PostgreSQL Library Management System

This guide is built around your own project setup:

- **Compose services:** db (PostgreSQL 15) and web (Django)
- **Database:** library_db **User:** quantumd
- **Django port:** 8000 **Postgres port (host):** 5433

Every command below is grouped so nothing repeats twice. Just find the section you need and copy-paste the command from your project's root folder (where `docker-compose.yml` lives).

Fastest way to start the whole project (first time):

1. Open a terminal in the `library-management-system` folder.
2. Run: `docker compose up --build`
3. In a second terminal, run migrations (see Section 3).
4. Open `http://localhost:8000` in your browser.

1. Docker Normal Commands

General Docker Engine commands. These work on any Docker setup — not tied to Compose or this project specifically.

Check that Docker is installed and see its version.

```
docker --version
```

See an overview of your Docker installation (containers, images, storage).

```
docker info
```

List currently running containers.

```
docker ps
```

List ALL containers, including stopped ones.

```
docker ps -a
```

List all images downloaded/built on your machine.

```
docker images
```

Delete a stopped container.

```
docker rm CONTAINER_NAME
```

Delete an image.

```
docker rmi IMAGE_NAME
```

Stop a running container.

```
docker stop CONTAINER_NAME
```

Start a previously stopped container.

```
docker start CONTAINER_NAME
```

Restart a container.

```
docker restart CONTAINER_NAME
```

See how much disk space Docker is using.

```
docker system df
```

Clean up unused containers, networks, and dangling images.

```
docker system prune
```

2. Docker Execution Commands

Run these from the project root (folder with docker-compose.yml). These control your db + web containers together as one project.

Build the web image from your Dockerfile (no start).

```
docker compose build
```

Start db + web in the foreground (logs shown live, Ctrl+C to stop).

```
docker compose up
```

Start db + web in the background (detached mode).

```
docker compose up -d
```

Rebuild the image and start everything — use this after editing Dockerfile or requirements.txt.

```
docker compose up --build
```

Show the status of this project's containers only.

```
docker compose ps
```

Stop the containers without removing them.

```
docker compose stop
```

Start previously stopped project containers again.

```
docker compose start
```

Restart both containers.

```
docker compose restart
```

Stop and remove containers + network (database data is kept).

```
docker compose down
```

Stop and remove containers + network + volumes (WARNING: this deletes your Postgres data too).

```
docker compose down -v
```

3. Docker Django Execution Commands

Use these to run Django's manage.py commands inside the already-running web container.

Apply database migrations to library_db.

```
docker compose exec web python manage.py migrate
```

Create new migration files after changing models.py.

```
docker compose exec web python manage.py makemigrations
```

Create an admin login for /admin.

```
docker compose exec web python manage.py createsuperuser
```

Open Django's interactive Python shell inside the container.

```
docker compose exec web python manage.py shell
```

Gather static files into the STATIC_ROOT folder.

```
docker compose exec web python manage.py collectstatic
```

Open a terminal (bash) inside the running web container.

```
docker compose exec web bash
```

Run a one-off manage.py command in a fresh temporary container (useful if 'web' isn't running yet).

```
docker compose run --rm web python manage.py CMD
```

4. Docker Django Logs Commands

Use these to see what Django is doing (errors, requests, print statements) inside the web container.

Show all logs from the web container since it started.

```
docker compose logs web
```

Follow web logs live, like tail -f (great while developing).

```
docker compose logs -f web
```

Show only the last 100 lines of web logs.

```
docker compose logs --tail=100 web
```

Same as above, but using the container's full name directly (works even if you're outside the project folder).

```
docker logs library-management-system-web-1
```

Follow logs live using the container's full name.

```
docker logs -f library-management-system-web-1
```

5. Docker Postgres User Log-in Commands

Use these to log into the PostgreSQL database itself and run SQL directly, using your project's credentials (user: quantumd, db: library_db).

Log into library_db as quantumd using psql (the Postgres shell).

```
docker compose exec db psql -U quantumd -d library_db
```

Same login, using the container's full name directly.

```
docker exec -it library-management-system-db-1 psql -U quantumd -d library_db
```

Open a plain terminal inside the db container (before running psql manually).

```
docker compose exec db bash
```

(Inside psql) List all databases.

```
\l
```

(Inside psql) List all tables in the current database.

```
\dt
```

(Inside psql) List all database users/roles.

```
\du
```

(Inside psql) Quit the psql shell.

```
\q
```

6. Docker PostgreSQL Logs Commands

Use these to see what the database container is doing — connections, errors, and startup status.

Show all logs from the db container since it started.

```
docker compose logs db
```

Follow Postgres logs live.

```
docker compose logs -f db
```

Show only the last 100 lines of Postgres logs.

```
docker compose logs --tail=100 db
```

Same as above, using the container's full name directly.

```
docker logs library-management-system-db-1
```

Quickly check if Postgres is up and accepting connections (used by your healthcheck).

```
docker compose exec db pg_isready -U quantumd -d library_db
```

7. Docker All Known Commands (Extra Reference)

Useful commands that don't fit the sections above — volumes, networks, and inspecting things inside your containers.

List all Docker volumes on your machine.

```
docker volume ls
```

See details about your Postgres data volume (where your DB data actually lives).

```
docker volume inspect library-management-system_postgres_data
```

List all Docker networks.

```
docker network ls
```

See full low-level details (IP, mounts, env vars) of a container.

```
docker inspect library-management-system-web-1
```

Copy a file out of a container to your current folder.

```
docker cp library-management-system-web-1:/app/somefile.txt .
```

Live view of CPU/RAM usage for all running containers.

```
docker stats
```

Validate and print the final, resolved docker-compose.yml.

```
docker compose config
```

Show the running processes inside each project container.

```
docker compose top
```

Open a shell inside any container, even without Compose commands.

```
docker exec -it library-management-system-web-1 sh
```

Tip: container names above assume the project folder is named `library-management-system`. Run `docker ps` any time to confirm the exact container names on your machine.